Thread Pool

Thread Pool · Cap concurrent thumbnail jobs · Build with Waqar

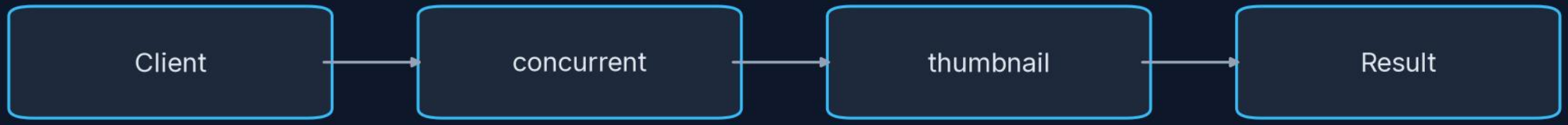
Today's Hook

- Today's force: Cap concurrent thumbnail jobs
- We earn Thread Pool by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: SimpleThreadPool

The Problem

- Scenario: Cap concurrent thumbnail jobs
- Constraints: solve «Cap concurrent thumbnail jobs» with Thread Pool — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Cap concurrent thumbnail jobs
- Success looks like: 5

Structure



Thread Pool: Cap concurrent thumbnail jobs

Real-World Scenario

- Running example: Cap concurrent thumbnail jobs
- Key types: SimpleThreadPool
- Happy-path stdout cue: 5
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.threadpool.problem;

import java.util.*;

public final class NaiveThumbs {
  public static int run(int n){ int c=0; for(int i=0;i<n;i++) c++; return c; } // unbounded threads in real life
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Cap concurrent thumbnail jobs
- Without Thread Pool, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.threadpool.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class SimpleThreadPool {
  private final ExecutorService pool;
  SimpleThreadPool(int n){ pool=Executors.newFixedThreadPool(n); }
  <T> Future<T> submit(Callable<T> c){ return pool.submit(c); }
  void shutdown(){ pool.shutdown(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.threadpool.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Thread Pool: cap thumbnail jobs ===");
    System.out.println("--- naive ---"); System.out.println(NaiveThumbs.run(5));
System.out.println("--- thread pool ---");
var pool=new SimpleThreadPool(2); List<Future<Integer>> fs=new ArrayList<>();
for(int i=0;i<5;i++){ int id=i; fs.add(pool.submit(() -> id*id)); }
System.out.println(fs.stream().map(f -> { try { return f.get(); } catch(Exception e){ throw new RuntimeException(e);} }).toList());
pool.shutdown();
  }
}
```

Walkthrough Output

```
=== Thread Pool: cap thumbnail jobs ===
--- naive ---
5
--- thread pool ---
[0, 1, 4, 9, 16]
```

Step-by-Step Trace

- Match stdout to stages — look for: 5
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 36-thread-pool ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Thread Pool
// Prefer records + sealed where they shrink the demo.
// Problem under test: Cap concurrent thumbnail jobs
```

Trade-offs

- Use Thread Pool when «Cap concurrent thumbnail jobs» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Thread Pool in product language.

Related Patterns

- Name the pattern usually confused with Thread Pool and separate the forces.
- Sketch the roles for «Cap concurrent thumbnail jobs» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Cap concurrent thumbnail jobs
- Naive pain → Thread Pool boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.